



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

**Faculty of Engineering (Degree)
Semester – 7th**

Subject Name – Modern Compiler Design

Subject Code – BE103701

Faculty Name – Radhika Gokani

CHAPTER -1

1. Overview of the Compiler and its Structure:

A compiler is a software tool that translates human-readable source code into machine-executable code. It acts as a translator, bridging the gap between high-level programming languages and the low-level instructions that computers can understand. The compiler's structure typically involves a front-end for source code analysis, a back-end for code generation, and often an optimizer in between.

OR

A **compiler** is a special program that converts source code written in a high-level programming language (like C, Java, or Python) into a low-level language (usually machine code or intermediate code) that a computer can execute. It performs analysis, optimization, and code generation to ensure that the resulting executable is efficient and correct.

Compiler Types

Type	Description
Single-pass	Goes through the code once; faster but less powerful.
Multi-pass	Multiple analysis passes for accuracy and optimization.
Cross compiler	Generates code for a different platform.
Just-in-time (JIT) compiler	Compiles code during runtime (used in JVM, .NET).

Structure of a Compiler

A compiler is typically broken down into several phases:

1. Lexical Analysis (Scanner)

- Converts source code into a stream of tokens.
- Removes whitespace and comments.
- Detects lexical errors.
- Uses a symbol table to store identifiers and keywords.

2. Syntax Analysis (Parser)

- Builds a **parse tree** or **abstract syntax tree (AST)** based on grammar rules.
- Ensures the code follows the syntactic structure of the language.
- Detects syntax errors.

3. Semantic Analysis

- Checks for semantic consistency (e.g., type compatibility).
- Validates declarations and usages.
- Annotates the AST with additional information.

4. Intermediate Code Generation

- Converts the AST into a lower-level, platform-independent intermediate representation (IR).
- Common formats include three-address code or Static Single Assignment (SSA) form.

5. Optimization

- Improves code efficiency without altering behavior.
- Examples: removing dead code, loop unrolling, constant folding.

6. Code Generation

- Translates optimized IR to target machine code or assembly code.
- Considers CPU architecture and instruction set.

7. Code Linking and Assembly

- Combines the object code with libraries and other modules.
- Produces the final executable.

Language processor

A **language processor** is a type of software system that converts source code written in a programming language into a form that can be understood and executed by a computer. Its job is to bridge the gap between human-readable code and machine-executable instructions.

Types of Language Processors

Processor Type	Role	Example
Compiler	Translates the entire source code into machine code before execution	GCC for C/C++
Interpreter	Translates and executes code line-by-line at runtime	Python Interpreter
Assembler	Converts assembly language into machine code	NASM
Linker	Combines multiple object files into a single executable	GNU LD
Loader	Loads executable files into memory for execution	OS-specific Loaders

Comparison: Compiler vs Interpreter

Feature	Compiler	Interpreter
Execution Speed	Faster (precompiled)	Slower (runtime translation)
Error Detection	All at once (after compilation)	Line-by-line
Output	Machine code	Immediate results

1.1 Applications of language processors

1. Software Development

- **Compilers and interpreters** translate human-written code into machine-readable formats.
- Enable programming in high-level languages like Java, Python, C++, etc.
- Support debugging and testing through syntax and semantic checks.

2. Embedded Systems

- Used to convert source code into highly optimized machine code for microcontrollers and embedded devices.
- Essential for IoT devices, automotive systems, medical equipment, etc.

3. Mobile App Development

- JIT compilers (used in Android's Dalvik and ART) boost performance by compiling code during runtime.
- Language processors are critical to handling platform-specific constraints.

4. Web Development

- Languages like JavaScript and Python rely on interpreters to execute code in browsers and servers.
- Language processors power dynamic web pages and backend systems.

5. Artificial Intelligence & Machine Learning

- Custom scripting languages and interpreters are used in data preprocessing and algorithm development.
- Examples include TensorFlow's XLA compiler for optimized code generation.

6. Education & Learning Platforms

- Language processors make it possible to build code playgrounds where students can practice coding in real time.
- Examples include platforms like Codecademy or repl.it..

7. Cross-Platform Development

- Cross-compilers enable building apps for different operating systems (e.g. compiling Windows apps on Linux).
- Useful in game development, enterprise apps, and tools that support multiple environments.

8. Runtime Optimization

- Interpreters and JIT compilers allow adaptive performance improvements by analyzing code during execution.
- Improves memory usage, execution speed, and resource management.

Real-World Tools That Use Language Processors

Tool	Processor Used	Purpose
GCC	Compiler	C/C++ program compilation
JVM	JIT Compiler + Interpreter	Java program execution
Python	Interpreter	Dynamic scripting
NASM	Assembler	Assembly code translation

1.2 Definition Structure-Working of compiler

Phase	Description
Lexical Analysis	Breaks input code into tokens (keywords, identifiers, operators).
Syntax Analysis	Constructs a parse tree to ensure code follows grammatical rules.
Semantic Analysis	Checks for logical consistency and data type correctness.
Intermediate Code Generation	Converts the code into an abstract form like three-address code.
Optimization	Enhances performance by removing redundancies and improving efficiency.
Code Generation	Converts intermediate code to machine code or assembly code.

Code Linking & Assembly	Links external libraries and creates the final executable file.
------------------------------------	---

⚙ Working of a Compiler: Step-by-Step

1. Lexical Analysis

- Reads the raw source code.
- Splits it into tokens using patterns (like regular expressions).
- Removes whitespace and comments.

2. Syntax Analysis

- Parses the token stream into a hierarchical structure.
- Detects syntax errors like missing semicolons or unmatched braces.

3. Semantic Analysis

- Ensures variables are declared before use.
- Checks type rules (like adding integer to a string? Nope!).

4. Intermediate Code Generation

- Produces a middle layer code that is easier to optimize and is platform-independent.

5. Code Optimization

- Improves speed and memory use.
- Examples: loop transformations, inlining, dead code removal.

6. Code Generation

- Converts optimized code to machine instructions tailored to the target CPU.

7. Linking & Assembly

- Combines the compiled code with libraries and other modules.
- Produces the final runnable program.

1.3 The science of building compilers

- The **science of building compilers** lies at the intersection of **theory, systems engineering, and linguistic analysis**—a true blend of logic and creativity.
- **overview of the scientific foundations:**

Area of Study	Key Topics
Lexical & Syntax Analysis	Regular expressions, context-free grammars, parsing trees
Semantic Analysis	Type checking, scope resolution, symbol tables
Intermediate Representations (IR)	Three-address code, control flow graphs
Optimization Techniques	Loop unrolling, dead code elimination, constant propagation
Code Generation & Emission	Target architecture, instruction selection
Error Handling & Recovery	Detecting and reporting syntactic/semantic issues

Compiler science is a subfield of **Computer Science** that focuses on designing and implementing compilers. It combines:

- **Formal Language Theory:** Understanding grammars, automata, and parsing techniques.
- **Program Analysis and Optimization:** Enhancing performance while preserving correctness.
- **Code Generation:** Translating abstract representations into executable code.

- **System-Level Engineering:** Integrating with hardware and operating systems.

1.4 Basic understanding of interpreter and assembler

Interpreter: Real-Time Translator

An **interpreter** is a type of language processor that directly executes instructions written in a high-level programming language **line-by-line**, without converting them into machine code first.

How It Works:

- Reads one line of source code.
- Translates and executes it immediately.
- Continues to the next line until the end.

Key Traits:

- Slower execution (since translation happens at runtime).
- Easier for debugging and testing.
- Common in scripting languages like **Python, JavaScript, Ruby, and MATLAB**.

Assembler: Bridge Between Assembly and Machine Code

An **assembler** is a language processor that translates **assembly language** (a low-level language close to machine instructions) into **machine code** (binary).

How It Works:

- Takes mnemonic assembly instructions (e.g., MOV A, B) and converts them into binary (e.g., 10110000).
- Produces **object code**, which is used by the CPU.

Key Traits:

- Works with specific CPU architectures.
- Needed in system programming, embedded systems, and hardware-level tasks.
- Examples: **NASM, MASM, GNU Assembler (GAS)**.

1.6 Difference between interpreter and compiler

Feature	Interpreter	Assembler
Input Language	High-level (like Python, JavaScript)	Low-level (Assembly language)
Output	Immediate execution of code	Converts to machine code (binary)
Execution Speed	Slower (line-by-line)	Faster (compiled before execution)
Debugging	Easier to debug	More complex, but powerful
Complexity	Easier to learn	Requires deep understanding
Use Cases	Scripting, web dev, data analysis	System programming, hardware tasks
Component	Interpreter	Assembler
Goal	Execute high-level source code	Convert low-level assembly to machine code
Role	Acts as a runtime translator	Acts as a compiler for assembly languages
Input	Human-readable high-level programs	Assembly instructions with mnemonics

Feature	Interpreter	Assembler
Output	Immediate execution (no separate file)	Object code or machine code (binary)

1.7 Compilation of source code into target language

deep dive into how **source code** written in a high-level programming language gets transformed into a **target language** (like machine code or bytecode) through compilation.

Compilation is the systematic conversion of source code (e.g., in C, C++, Java) into a lower-level representation that can be executed by a computer. This target language is typically:

- **Machine code** (binary instructions for the processor)
- **Bytecode** (for virtual machines like JVM or .NET CLR)
- **Assembly code** (a readable form close to machine instructions)

Stages of Compilation

1. Lexical Analysis (Scanning)

- Breaks source code into **tokens**: keywords, identifiers, literals.
- Removes comments and whitespace.
- Builds a symbol table.

→ *Example*: `int x = 10;` becomes tokens: `int, x, =, 10, ;`

2. Syntax Analysis (Parsing)

- Constructs a **parse tree** or **abstract syntax tree (AST)** using grammar rules.
- Verifies correct structure (e.g., proper nesting of loops, blocks).

→ Checks: Is the if-statement correctly formed?

3. Semantic Analysis

- Ensures type consistency, variable declaration, function signatures.
- Annotates AST with additional meaning for later stages.

→ *Error Example*: Using a string where an integer is expected.

4. Intermediate Code Generation

- Produces a lower-level, platform-independent code (like three-address code or LLVM IR).
- Easier to optimize and versatile across architectures.

→ *Example*:

`t1 = 10`

`x = t1`

5. Code Optimization

- Removes redundant code and improves performance.
- Techniques include: dead code elimination, loop unrolling, constant folding.

→ *Turns*:

`x = 5`

Into:

`x = 5`

6. Target Code Generation

- Converts optimized intermediate code into machine code or bytecode.
- Tailored for specific CPU instructions (x86, ARM, etc.)

→ *Machine Code Example* (binary or hexadecimal):

`MOV AX, 10`

7. Assembly and Linking

- Assembler converts assembly into object code.
- Linker joins object files and libraries into a complete executable.

→ Final product: .exe, .out, or .class file

Target Languages

Target Language	Where It's Used
Machine Code	Native apps, operating systems
Bytecode	Java, .NET applications
Assembly Language	Low-level embedded programs

1.8 Cousins of compiler

the compiler's extended family—let's meet the “**cousins**” in the language processing world! These are tools that perform related but distinct tasks in translating, executing, or analyzing code.

Cousins of a Compiler

1. Interpreter

- Directly executes source code line-by-line without converting to machine code beforehand.
- Slower than compilers but great for rapid testing and development.

Used in: Python, Ruby, JavaScript (traditionally)

2. Assembler

- Converts **assembly language** (a low-level, readable form of machine instructions) into **machine code**.
- Acts as a translator between human-readable low-level code and binary.

Used in: Low-level system programming, embedded systems

3. Linker

- Combines multiple **object files** into a single executable.
- Resolves external references, like function calls across files.

Used in: Almost every compiled language workflow (e.g., C, C++)

4. Loader

- Loads an executable program into memory and prepares it for execution.
- Handles memory allocation, initialization, and execution flow.

Used in: OS-level program execution

5. Preprocessor

- Runs before compilation to handle directives like macro expansion, file inclusion, and conditional compilation.

Used in: C/C++ (#include, #define, #ifdef etc.)

6. Just-In-Time (JIT) Compiler

- A hybrid of compiler and interpreter: compiles bytecode into machine code during runtime.
- Combines flexibility of interpretation with speed of compilation.

Used in: Java (JVM), .NET (CLR)

Comparison Snapshot

Tool	Input	Output	Role
Compiler	High-level code	Machine code	Translates entire program
Interpreter	High-level code	Immediate result	Executes line-by-line
Assembler	Assembly code	Machine code	Translates low-level code
Linker	Object files	Executable	Combines code modules
Loader	Executable	In-memory image	Prepares to run
Preprocessor	Source code	Modified source	Handles directives
JIT Compiler	Bytecode	Machine code	Runtime compilation

1.9 Types of compiler

Compilers come in several flavors, each tailored for different needs and workflows in programming. Think of them like specialists in a medical team—each one excels at a different function but ultimately serves the same goal: converting your code into something the machine understands.

Types of Compilers

1. Single-Pass Compiler

- Processes source code only once.
- Faster and simpler, but limited in optimization.
- Suited for smaller, straightforward programs.

Used in: early or embedded systems

2. Multi-Pass Compiler

- Reads the source code multiple times to build more accurate syntax trees and apply deeper optimizations.
- Slower but more thorough and powerful.

Used in: production-level C/C++, Java compilers

3. Cross Compiler

- Generates code for a different platform than the one it runs on.
- Useful for building programs for embedded systems or different OS.

Example: A Windows-based compiler producing code for ARM architecture.

4. Bootstrap Compiler

- A self-compiling compiler: written in the same language it compiles.
- Helps port languages to new platforms.

Example: A C compiler written in C.

5. Just-In-Time (JIT) Compiler

- Compiles code during runtime rather than beforehand.
- Balances flexibility (like interpreters) with performance (like compilers).

Used in: Java (JVM), .NET CLR, Python (via PyPy)

6. Incremental Compiler

- Compiles only the parts of the program that changed.
- Speeds up development for large projects.

Used in: modern IDEs like IntelliJ, Eclipse

7. Ahead-of-Time (AOT) Compiler

- Compiles source or intermediate code into machine code *before* runtime.
- Opposite of JIT—often used for performance optimization.

Used in: Angular apps (via ngc), LLVM

Categories by Function

Category	Example Compiler	Role
Language-specific	javac, gcc	Targets a specific language
Source-to-source	TypeScript → JS	Converts one high-level lang to another
Intermediate code	LLVM IR, Java Bytecode	Creates platform-neutral code
